```
// ===== lib/main.dart =====
import 'dart:async';
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:window_manager/window_manager.dart';

import 'app/app.dart';
import 'app/bootstrap.dart';
import 'app/platform/desktop_shell.dart';
import 'app/system_bars.dart';
import 'core/logging/app_logger.dart';
import 'core/storage/path_manager.dart';
import 'domain/update/distribution.dart';

/// 单实例通道：当第二个实例启动并检测到已有实例运行时，原生层会向本实例
/// 发送 "showMainWindow" 调用
const MethodChannel _singleInstanceChannel = MethodChannel(
'bili_audio/single_instance',
);

Future<void> _initSingleInstanceChannel() async {
_singleInstanceChannel.setMethodCallHandler((call) async {
if (call.method == 'showMainWindow') {
await windowManager.show();
await windowManager.focus();
}
});
}

void main() {
BindingBase.debugZoneErrorsAreFatal = true;
WidgetsFlutterBinding.ensureInitialized();
final rootZone = Zone.current;

runZonedGuarded(
() {
FlutterError.onError = (details) {
FlutterError.presentError(details);
final where = '${details.context ?? ''}';
final building = where.contains('building');
AppLogger.e(
building ? 'page build failed ($where)' : details.exceptionAsString(),
tag: building ? 'nav' : 'flutter',
error: details.exception,
stackTrace: details.stack,
);
```

```
};

PlatformDispatcher.instance.onError = (error, stack) {
AppLogger.e(
'platform error',
tag: 'flutter',
error: error,
stackTrace: stack,
);
rootZone.run(() => _showBootError(error, stack));
return true;
};

_runApp(rootZone).catchError((Object e, StackTrace st) {
AppLogger.e('boot failed', tag: 'boot', error: e, stackTrace: st);
rootZone.run(() => _showBootError(e, st));
});
},
(error, stack) {
AppLogger.e('zone error', tag: 'boot', error: error, stackTrace: stack);
rootZone.run(() => _showBootError(error, stack));
},
);
}

void _showBootError(Object error, StackTrace? stack) {
try {
runApp(_BootErrorScreen(error: error, stack: stack));
} catch (caught, st) {
AppLogger.e(
'unable to show error screen',
tag: 'boot',
error: caught,
stackTrace: st,
);
}
}

Future<void> _runApp(Zone rootZone) async {
AppLogger.i('main entered, kIsWeb=$kIsWeb', tag: 'boot');
await enableImmersiveSystemBars();

// 注册单实例通道
await _initSingleInstanceChannel();

// 初始化路径管理与日志文件
await PathManager.init();
await AppLogger.init(PathManager.logsDir);

// 桌面端窗口设置
```

```
if (!kIsWeb && (Platform.isWindows || Platform.isMacOS || Platform.isLinux)) {
AppLogger.i('initializing window_manager', tag: 'boot');
await windowManager.ensureInitialized();

const WindowOptions windowOptions = WindowOptions(
size: Size(1280, 800),
minimumSize: Size(360, 640),
center: true,
backgroundColor: Colors.transparent,
skipTaskbar: false,
titleBarStyle: TitleBarStyle.hidden,
title: productName,
);

await windowManager.waitUntilReadyToShow(windowOptions, () async {
await windowManager.show();
await windowManager.focus();
});
AppLogger.i('window shown', tag: 'boot');

await initDesktopShell();
AppLogger.i('tray initialized', tag: 'boot');
}

// 数据库 / 网络 / 播放器等全局依赖 (M6)
final overrides = await bootstrap();
AppLogger.i('dependencies ready', tag: 'boot');

rootZone.run(() {
runApp(ProviderScope(overrides: overrides, child: const BiliAudioApp()));
});
AppLogger.i('runApp returned', tag: 'boot');
}

/// 启动期致命错误显示页
class _BootErrorScreen extends StatelessWidget {
const _BootErrorScreen({required this.error, this.stack});

final Object error;
final StackTrace? stack;

@override
Widget build(BuildContext context) {
return MaterialApp(
debugShowCheckedModeBanner: false,
home: AnnotatedRegion<SystemUiOverlayStyle>(
value: immersiveOverlayStyle(Brightness.dark),
child: Scaffold(
backgroundColor: const Color(0xFF1A1A1A),
body: SafeArea(
```

```
child: Padding(
padding: const EdgeInsets.all(24),
child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
children: [
const Row(
children: [
Icon(
Icons.error_outline,
color: Color(0xFFFF6B6B),
size: 28,
),
SizedBox(width: 12),
Text(
'启动失败 / Startup Failed',
style: TextStyle(
color: Color(0xFFFF6B6B),
fontSize: 20,
fontWeight: FontWeight.w700,
),
),
],
),
const SizedBox(height: 16),
const Text(
'错误信息:',
style: TextStyle(
color: Color(0xFFB0B0B0),
fontSize: 12,
fontWeight: FontWeight.w600,
),
),
const SizedBox(height: 6),
SelectableText(
error.toString(),
style: const TextStyle(
color: Colors.white,
fontSize: 14,
fontFamily: 'monospace',
),
),
if (stack != null) ...[
const SizedBox(height: 16),
const Text(
'堆栈:',
style: TextStyle(
color: Color(0xFFB0B0B0),
fontSize: 12,
fontWeight: FontWeight.w600,
),
```

```
),
const SizedBox(height: 6),
Expanded(
child: SingleChildScrollView(
child: SelectableText(
stack.toString(),
style: const TextStyle(
color: Color(0xFFB0B0B0),
fontSize: 11,
fontFamily: 'monospace',
),
),
),
),
],
],
),
),
),
),
),
);
}
}

// ===== lib/app/adaptive/layout_settings.dart =====
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../data/db/app_database.dart';
import '../providers.dart';
import 'system_car.dart';

/// 设置里的车机布局开关。自动只在系统报告车载界面时生效。
enum CarLayoutMode {
auto,
on,
off;

static CarLayoutMode parse(String? raw) => switch (raw) {
'on' => CarLayoutMode.on,
'off' => CarLayoutMode.off,
_ => CarLayoutMode.auto,
};

String get storageValue => name;
}

/// 分隔比例与车机开关。车机是否显示还要看 [systemCar]。
class LayoutSettings {
const LayoutSettings({
```

```
this.splitRatio = 0.62,
this.carMode = CarLayoutMode.auto,
this.systemCar = false,
});

final double splitRatio;
final CarLayoutMode carMode;
final bool systemCar;

bool get carLayout => switch (carMode) {
CarLayoutMode.on => true,
CarLayoutMode.off => false,
CarLayoutMode.auto => systemCar,
};

LayoutSettings copyWith({
double? splitRatio,
CarLayoutMode? carMode,
bool? systemCar,
}) =>
LayoutSettings(
splitRatio: splitRatio ?? this.splitRatio,
carMode: carMode ?? this.carMode,
systemCar: systemCar ?? this.systemCar,
);
}

/// 纯函数，便于单测：宽度不能单独打开车机布局。
bool resolveCarLayout(CarLayoutMode mode, bool systemCar) =>
LayoutSettings(carMode: mode, systemCar: systemCar).carLayout;

class LayoutSettingsNotifier extends Notifier<LayoutSettings> {
var _epoch = 0;

@override
LayoutSettings build() {
_load();
return const LayoutSettings();
}

Future<void> _load() async {
final token = ++_epoch;
final db = ref.read(databaseProvider);
final ratioRaw = await db.settingsDao.get(_splitKey);
final modeRaw = await db.settingsDao.get(_carKey);
final system = await loadSystemCarUi();
if (token != _epoch || !ref.mounted) return;
final ratio = double.tryParse(ratioRaw ?? '') ?? 0.62;
state = LayoutSettings(
splitRatio: ratio.clamp(0.4, 0.75),
```

```
carMode: CarLayoutMode.parse(modeRaw),
systemCar: system,
);
}

Future<void> setSplitRatio(double ratio) => _write(
state.copyWith(splitRatio: ratio.clamp(0.4, 0.75)),
(db) => db.settingsDao.set(_splitKey, ratio.clamp(0.4, 0.75).toString()),
);

Future<void> setCarMode(CarLayoutMode mode) => _write(
state.copyWith(carMode: mode),
(db) => db.settingsDao.set(_carKey, mode.storageValue),
);

Future<void> _write(
LayoutSettings next,
Future<void> Function(AppDatabase db) persist,
) async {
_epoch++;
state = next;
final db = ref.read(databaseProvider);
await persist(db);
}
}

const _splitKey = 'layout.split_ratio';
const _carKey = 'layout.car_mode';

final layoutSettingsProvider =
NotifierProvider<LayoutSettingsNotifier, LayoutSettings>(
LayoutSettingsNotifier.new,
);

// ===== lib/app/adaptive/max_width_box.dart =====
import 'dart:math' as math;

import 'package:flutter/material.dart';

/// 表单和设置在宽窗口上居中，并不超过 [maxWidth]。
class MaxWidthBox extends StatelessWidget {
const MaxWidthBox({required this.child, this.maxWidth = 640, super.key});

final Widget child;
final double maxWidth;

@override
Widget build(BuildContext context) {
return LayoutBuilder(
builder: (context, constraints) {
```

```
final width = math.min(maxWidth, constraints.maxWidth);
final height = constraints.maxHeight;
return Align(
alignment: Alignment.topCenter,
child: SizedBox(width: width, height: height, child: child),
);
},
);
}
}

// ===== lib/app/adaptive/player_shortcuts.dart =====
import 'package:flutter/services.dart';
import 'package:flutter/widgets.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../data/player/bili_audio_handler.dart';
import '../providers.dart';
import 'layout_settings.dart';

final searchFocusNodeProvider = Provider<FocusNode>((ref) {
final node = FocusNode(debugLabel: 'search');
ref.onDispose(node.dispose);
return node;
});

void togglePlay(BiliAudioHandler handler) {
if (handler.engine.playing) {
handler.pause();
} else {
handler.play();
}
}

void seekBy(BiliAudioHandler handler, Duration delta) {
final total = handler.mediaItem.hasValue ? handler.mediaItem.value?.duration : null;
var next = handler.engine.position + delta;
if (next < Duration.zero) next = Duration.zero;
if (total != null && next > total) next = total;
handler.seek(next);
}

bool textFieldFocused() {
final focus = FocusManager.instance.primaryFocus;
final context = focus?.context;
if (context == null) return false;
return context.findAncestorStateOfType<EditableTextState>() != null;
}
```

```
void focusSearch(BuildContext context, WidgetRef ref) {
if (ref.read(layoutSettingsProvider).carLayout) return;
GoRouter.of(context).go('/search');
WidgetsBinding.instance.addPostFrameCallback((_) {
ref.read(searchFocusNodeProvider).requestFocus();
});
}

/// 桌面和 iPad 外接键盘。焦点在输入框时不拦截。
class PlayerShortcutLayer extends ConsumerStatefulWidget {
const PlayerShortcutLayer({required this.child, super.key});

final Widget child;

@override
ConsumerState<PlayerShortcutLayer> createState() => _PlayerShortcutLayerState();
}

class _PlayerShortcutLayerState extends ConsumerState<PlayerShortcutLayer> {
@override
void initState() {
super.initState();
HardwareKeyboard.instance.addHandler(_onKey);
}

@override
void dispose() {
HardwareKeyboard.instance.removeHandler(_onKey);
super.dispose();
}

bool _onKey(KeyEvent event) {
if (event is! KeyDownEvent || textFieldFocused()) return false;
final key = event.logicalKey;
final pressed = HardwareKeyboard.instance.logicalKeysPressed;
final command = pressed.contains(LogicalKeyboardKey.controlLeft) ||
pressed.contains(LogicalKeyboardKey.controlRight) ||
pressed.contains(LogicalKeyboardKey.metaLeft) ||
pressed.contains(LogicalKeyboardKey.metaRight);
if (command && key == LogicalKeyboardKey.keyF) {
focusSearch(context, ref);
return true;
}
final handler = ref.read(audioHandlerProvider);
if (key == LogicalKeyboardKey.space) {
togglePlay(handler);
return true;
}
if (key == LogicalKeyboardKey.arrowLeft) {
seekBy(handler, const Duration(seconds: -5));
```

```
return true;
}
if (key == LogicalKeyboardKey.arrowRight) {
seekBy(handler, const Duration(seconds: 5));
return true;
}
if (key == LogicalKeyboardKey.arrowUp ||
key == LogicalKeyboardKey.mediaTrackPrevious) {
handler.skipToPrevious();
return true;
}
if (key == LogicalKeyboardKey.arrowDown ||
key == LogicalKeyboardKey.mediaTrackNext) {
handler.skipToNext();
return true;
}
return false;
}

@override
Widget build(BuildContext context) => widget.child;
}

// ===== lib/app/adaptive/split_view.dart =====
import 'package:flutter/material.dart';

import '../widgets/region_surface.dart';

/// 主从栏分隔。比例是左栏占可用宽度的份额，限制在 0.4–0.75。
class SplitView extends StatefulWidget {
const SplitView({
required this.ratio,
required this.onRatioEnd,
required this.first,
required this.second,
super.key,
});

final double ratio;
final ValueChanged<double> onRatioEnd;
final Widget first;
final Widget second;

static double clampRatio(double ratio) => ratio.clamp(0.4, 0.75);

@override
State<SplitView> createState() => _SplitViewState();
}

class _SplitViewState extends State<SplitView> {
```

```
double? _dragging;

@override
Widget build(BuildContext context) {
final ratio = SplitView.clampRatio(_dragging ?? widget.ratio);
return LayoutBuilder(
builder: (context, constraints) {
const handleWidth = 12.0;
final available = constraints.maxWidth;
if (available <= 0) return const SizedBox.shrink();
final firstWidth = available * ratio;
final handleLeft = (firstWidth - handleWidth / 2).clamp(
0.0,
(available - handleWidth).clamp(0.0, double.infinity),
);
return Stack(
fit: StackFit.expand,
children: [
Row(
children: [
SizedBox(width: firstWidth, child: widget.first),
Expanded(
child: RegionSurface(
edge: RegionEdge.leading,
child: widget.second,
),
),
],
),
Positioned(
left: handleLeft,
top: 0,
bottom: 0,
width: handleWidth,
child: MouseRegion(
cursor: SystemMouseCursors.resizeColumn,
child: GestureDetector(
key: const Key('split-handle'),
behavior: HitTestBehavior.opaque,
onHorizontalDragUpdate: (details) {
final next = firstWidth + details.delta.dx;
setState(
() => _dragging = SplitView.clampRatio(next / available),
);
},
onHorizontalDragEnd: (_) {
final settled = SplitView.clampRatio(
_dragging ?? widget.ratio,
);
setState(() => _dragging = null);
```

```
widget.onRatioEnd(settled);
},
child: const SizedBox.expand(),
),
),
),
],
);
},
);
}
}

// ===== lib/app/adaptive/system_car.dart =====
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';

const systemCarChannel = MethodChannel('bili_audio/ui_mode');

/// Android 车载 UI 模式。其他平台、读不到或调用失败时为 false。
Future<bool> loadSystemCarUi() async {
if (kIsWeb || !Platform.isAndroid) return false;
try {
final value = await systemCarChannel.invokeMethod<bool>('isCar');
return value ?? false;
} catch (e, st) {
debugPrint('ui_mode isCar failed: $e\n$st');
return false;
}
}

// ===== lib/app/adaptive/window_metrics.dart =====
import 'package:flutter/material.dart';

/// 窗口宽度档。布局只看逻辑像素，不看设备类型。
///
/// large 从 1194 起：11 英寸 iPad 横屏是 1194，任务要求这一档用主从栏。
enum WindowWidth { compact, medium, expanded, large, extraLarge }

/// 窗口高度档。只影响纵向密度，不单独决定导航形态。
enum WindowHeight { short, regular, tall }

/// 由内容区宽高得到的布局档位。
class WindowMetrics {
const WindowMetrics(this.width, this.height);

factory WindowMetrics.fromSize(Size size) => WindowMetrics(
widthClass(size.width),
```

```
heightClass(size.height),
);

final WindowWidth width;
final WindowHeight height;

bool get usesRail => width != WindowWidth.compact && !isShort;

bool get usesMasterDetail =>
(width == WindowWidth.large || width == WindowWidth.extraLarge) &&
!isShort;

bool get isShort => height == WindowHeight.short;

/// 矮窗口（手机横屏）用底部导航，避免侧栏在高度不足时溢出。
bool get usesBottomNav => !usesRail && !usesMasterDetail;

@override
bool operator ==(Object other) =>
other is WindowMetrics && other.width == width && other.height == height;

@override
int get hashCode => Object.hash(width, height);
}

WindowWidth widthClass(double width) {
if (width < 600) return WindowWidth.compact;
if (width < 840) return WindowWidth.medium;
if (width < 1194) return WindowWidth.expanded;
if (width < 1600) return WindowWidth.large;
return WindowWidth.extraLarge;
}

WindowHeight heightClass(double height) {
if (height < 500) return WindowHeight.short;
// 390×844 要算 tall，1280×800 与 1194×834 仍是 regular。
if (height < 844) return WindowHeight.regular;
return WindowHeight.tall;
}

/// 壳层把内容区档位传给子页面。播放页等壳外路由自行用 [WindowMetrics.fromSize]。
class WindowMetricsScope extends InheritedWidget {
const WindowMetricsScope({
required this.metrics,
required super.child,
super.key,
});

final WindowMetrics metrics;
```

```
static WindowMetrics? maybeOf(BuildContext context) =>
context.dependOnInheritedWidgetOfExactType<WindowMetricsScope>()?.metrics;

@override
bool updateShouldNotify(WindowMetricsScope oldWidget) =>
metrics != oldWidget.metrics;
}

// ===== lib/app/app.dart =====
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../domain/update/distribution.dart';
import '../l10n/l10n.dart';
import 'adaptive/player_shortcuts.dart';
import 'platform/window_frame.dart';
import 'providers.dart';
import 'router.dart';
import 'system_bars.dart';
import 'theme.dart';
import 'widgets/window_title_bar.dart';

bool get isDesktop =>
!kIsWeb && (Platform.isWindows || Platform.isMacOS || Platform.isLinux);

class BiliAudioApp extends ConsumerWidget {
const BiliAudioApp({super.key});

@override
Widget build(BuildContext context, WidgetRef ref) {
final themeState = ref.watch(themeControllerProvider);
return MaterialApp.router(
onGenerateTitle: (context) => productTitle(),
localizationsDelegates: AppLocalizations.localizationsDelegates,
supportedLocales: AppLocalizations.supportedLocales,
locale: const Locale('zh'),
debugShowCheckedModeBanner: false,
theme: buildTheme(themeState.seed, Brightness.light),
darkTheme: buildTheme(themeState.seed, Brightness.dark),
themeMode: themeState.mode,
routerConfig: appRouter,
// 桌面端套一层无边框窗口外壳：8px 圆角 + 自定义标题栏
builder: (context, child) {
Widget content = PlayerShortcutLayer(
child: child ?? const SizedBox.shrink(),
```

```
);
if (isDesktop) content = _DesktopWindowShell(child: content);
if (!kIsWeb && Platform.isMacOS) {
content = _MacPlaybackMenu(child: content);
}
if (platformDrawsBehindSystemBars) {
content = AnnotatedRegion<SystemUiOverlayStyle>(
value: immersiveOverlayStyle(Theme.of(context).brightness),
child: content,
);
}
return content;
},
);
}
}

/// 桌面端窗口外壳（沿用 talix_companion_flutter 的无边框方案）。
///
/// 窗口背景在 main() 中设为透明，这里用 ClipRRect 裁出圆角。
/// 顶部标题栏可拖拽。Windows / Linux 在右侧放窗体按钮；
/// macOS 保留系统红绿灯，标题栏与红绿灯同高，标题靠右并垂直居中。
class _DesktopWindowShell extends StatelessWidget {
const _DesktopWindowShell({required this.child});

final Widget child;

@override
Widget build(BuildContext context) {
return ValueListenableBuilder<bool>(
valueListenable: desktopWindowRounded,
builder: (context, rounded, _) => ClipRRect(
borderRadius: BorderRadius.circular(rounded ? 8 : 0),
clipBehavior: Clip.antiAlias,
child: ColoredBox(
color: Theme.of(context).colorScheme.surface,
child: Stack(
clipBehavior: Clip.none,
children: [
Padding(
padding: EdgeInsets.only(top: WindowTitleBar.platformBarHeight),
child: child,
),
const Positioned(
top: 0,
left: 0,
right: 0,
child: WindowTitleBar(),
),
],
```

```
),
),
),
);
}
}

class _MacPlaybackMenu extends ConsumerWidget {
const _MacPlaybackMenu({required this.child});

final Widget child;

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final handler = ref.read(audioHandlerProvider);
return PlatformMenuBar(
menus: [
PlatformMenu(
label: productTitle(),
menus: [
PlatformMenuItem(
label: l10n.actionPlay,
onSelected: () => togglePlay(handler),
),
PlatformMenuItem(label: '上一首', onSelected: handler.skipToPrevious),
PlatformMenuItem(label: '下一首', onSelected: handler.skipToNext),
PlatformMenuItem(
label: l10n.settingsTitle,
onSelected: () => context.go('/settings'),
),
],
),
],
child: child,
);
}
}

// ===== lib/app/bootstrap.dart =====
import 'dart:async';
import 'dart:io';

import 'package:audio_service/audio_service.dart';
import 'package:dio/dio.dart';
import 'package:drift/drift.dart' show Value;
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/misc.dart' show Override;

import '../core/logging/app_logger.dart';
```

```
import '../core/network/bili_http.dart';
import '../core/network/interceptors/http_log_interceptor.dart';
import '../core/network/buvid_manager.dart';
import '../core/network/interceptors/header_interceptor.dart';
import '../core/network/wbi_key_manager.dart';
import '../core/storage/path_manager.dart';
import '../core/storage/secure_store.dart';
import '../core/utils/cover_url.dart';
import '../data/audio/audio_cache_store.dart';
import '../data/audio/cache_download_manager.dart';
import '../data/bili/bili_login_api.dart';
import '../data/bili/bili_ranking_api.dart';
import '../data/bili/bili_search_api.dart';
import '../data/bili/bili_stream_api.dart';
import '../data/bili/bili_video_api.dart';
import '../data/db/app_database.dart' hide Track;
import '../data/album/album_manager.dart';
import '../data/album/itunes_album_provider.dart';
import '../data/album/netease_album_provider.dart';
import '../data/album/qq_album_provider.dart';
import '../data/lyrics/bili_subtitle_provider.dart';
import '../data/lyrics/kugou_lyrics_provider.dart';
import '../data/lyrics/kuwo_lyrics_provider.dart';
import '../data/lyrics/lrclib_provider.dart';
import '../data/lyrics/lyrics_manager.dart';
import '../data/lyrics/netease_lyrics_provider.dart';
import '../data/lyrics/qq_lyrics_provider.dart';
import '../data/player/audio_engine.dart';
import '../data/player/media_library.dart';
import '../data/player/bili_audio_handler.dart';
import '../data/player/queue_snapshot_store.dart';
import '../data/era/netease_era_catalog.dart';
import '../data/repositories/bili_repositories.dart';
import '../data/repositories/stream_url_manager.dart';
import '../domain/entities/audio_track.dart';
import '../domain/update/distribution.dart';
import '../domain/title/title_cleaner.dart';
import 'platform/app_quit.dart';
import 'platform/desktop_platform.dart';
import 'platform/desktop_shell.dart';
import 'platform/window_frame.dart';
import 'theme.dart';
import '../data/update/app_version_source.dart';
import '../data/update/local_update_installer.dart';
import '../data/update/release_source.dart';
import '../features/album/album_controller.dart';
import '../features/login/login_controller.dart';
import '../features/lyrics/lyrics_controller.dart';
import '../features/settings/update_controller.dart';
import 'providers.dart';
```

```
/// 构建真实依赖并返回 ProviderScope overrides (M6)
///
/// 要求 [PathManager.init] 已完成。失败时抛出，由 main 展示启动错误页。
Future<List<Override>> bootstrap() async {
unawaited(deletePreviousMacAppBundles());
await initDesktopPlatform();
AppLogger.i(
'desktop ready os=${Platform.operatingSystem} smtc=$smtcAvailable',
tag: 'boot',
);

final db = AppDatabase();
AppLogger.i('database opened ${PathManager.databaseFile.path}', tag: 'boot');

final SecureStore store = FlutterSecureStore();

// 获取 buvid / WBI key 的请求不经过 BiliHttp 拦截器链，避免递归
final raw = Dio(BaseOptions(baseUrl: 'https://api.bilibili.com'))
..interceptors.add(HeaderInterceptor())
..interceptors.add(HttpLogInterceptor());
final http = BiliHttp(
store: store,
buvidManager: BuvidManager(store, () async {
final res = await raw.get<Map<String, dynamic>>('/x/frontend/finger/spi');
final data = res.data!['data'] as Map<String, dynamic>;
return (data['b_3'] as String, data['b_4'] as String);
}),
wbiKeyManager: WbiKeyManager(() async {
// 未登录时 code=-101，但 wbi_img 仍然返回
final res = await raw.get<Map<String, dynamic>>('/x/web-interface/nav');
final img =
(res.data!['data'] as Map<String, dynamic>)['wbi_img']
as Map<String, dynamic>;
return (img['img_url'] as String, img['sub_url'] as String);
}),
);

final cleaner = TitleCleaner.fromJson(
await rootBundle.loadString('assets/config/title_cleaning_rules.json'),
);

final customDir = await db.settingsDao.getCacheDir();
final downloads = CacheDownloadManager(
AudioCacheStore(
db,
customDir == null ? PathManager.audioDir : Directory(customDir),
limitBytes: db.settingsDao.getCacheMaxSize,
),
Dio(),
```

```
);

final qualityId = await db.settingsDao.getDefaultQuality();
final prefs = PlaybackPrefs(
quality: AudioQuality.values.firstWhere(
(q) => q.id == qualityId,
orElse: () => AudioQuality.k132,
),
loggedIn: (await store.readCookies())?.contains('SESSDATA') ?? false,
);
AppLogger.i(
'playback quality=${prefs.quality.name}(${prefs.quality.id}) '
'loggedIn=${prefs.loggedIn}',
tag: 'boot',
);

BiliAudioHandler build() => BiliAudioHandler(
engine: JustAudioEngine(),
streams: BiliStreamRepository(StreamUrlManager(db, BiliStreamApi(http))),
downloads: downloads,
snapshots: QueueSnapshotStore(db),
prepareTrack: (track) => db.tracksDao.ensureTrack(_trackRow(track)),
recorder: (track) async {
await db.tracksDao.ensureTrack(_trackRow(track));
final id = await db.historyDao.start(track.bvid, track.cid);
return (Duration p) =>
unawaited(db.historyDao.updateProgress(id, p, track.duration));
},
quality: () => prefs.quality,
loggedIn: () => prefs.loggedIn,
);

// audio_service 不支持 Windows/Linux (D3): 直接构造, Windows 另接 SMTC
final BiliAudioHandler handler;
if (Platform.isWindows || Platform.isLinux) {
handler = build();
if (Platform.isWindows && smtcAvailable) WindowsMediaBridge(handler);
} else {
handler = await AudioService.init(
builder: build,
config: const AudioServiceConfig(
androidNotificationChannelId: 'com.bilimusic.bili_audio.playback',
androidNotificationChannelName: '音频播放',
androidStopForegroundOnPause: false,
),
);
}
handler.browseLibrary = MediaLibrary(
db,
() => handler.playbackQueue.orderedTracks,
```

```
);
final restored = await handler.restore();
AppLogger.i('queue restored=$restored', tag: 'boot');

if (Platform.isWindows || Platform.isMacOS || Platform.isLinux) {
await restoreWindowFrame(db.settingsDao);
WindowFrameStore.attach(db.settingsDao);
await attachDesktopPlayback(handler);
}
// macOS 控制中心的播放/暂停/上下首/进度由 audio_service 写到
// MPNowPlayingInfoCenter。托盘、Windows 任务栏按钮和程序坞菜单在上面接上。

final initialTheme = ThemeState.decode(
await db.settingsDao.getThemeMode(),
await db.settingsDao.getThemeSeed(),
);

final lyricsManager = LyricsManager(
db,
PathManager.lyricsDir,
providers: [
BiliSubtitleProvider(api: http.dio),
NeteaseLyricsProvider(),
QqLyricsProvider(),
KugouLyricsProvider(),
KuwoLyricsProvider(),
LrclibProvider(),
],
cleaner: cleaner,
);
final albumManager = AlbumManager(
PathManager.albumDir,
providers: [
NeteaseAlbumProvider(),
QqAlbumProvider(),
ItunesAlbumProvider(),
],
cleaner: cleaner,
);

final videoApi = BiliVideoApi(http);

return [
databaseProvider.overrideWithValue(db),
lyricsManagerProvider.overrideWithValue(lyricsManager),
albumManagerProvider.overrideWithValue(albumManager),
loginApiProvider.overrideWithValue(BiliLoginApi(http, store)),
initialThemeProvider.overrideWithValue(initialTheme),
themeSaverProvider.overrideWithValue((theme) async {
final (mode, seed) = theme.encode();
```

```
await db.settingsDao.setBatch({'theme_mode': mode, 'theme_seed': seed});
}),
playbackPrefsProvider.overrideWithValue(prefs),
searchRepositoryProvider.overrideWithValue(
BiliSearchRepository(
BiliSearchApi(http),
cleaner,
pageList: videoApi.getPageList,
ranking: BiliRankingApi(http),
eraCatalog: NeteaseEraCatalog(),
),
),
videoRepositoryProvider.overrideWithValue(
BiliVideoRepository(db, videoApi, cleaner),
),
audioHandlerProvider.overrideWithValue(handler),
appVersionSourceProvider.overrideWithValue(const PackageAppVersionSource()),
if (offersInAppUpdate()) ...[
releaseSourceProvider.overrideWithValue(GithubReleaseSource()),
updateInstallerProvider.overrideWithValue(LocalUpdateInstaller()),
],
appQuitProvider.overrideWithValue(quitDesktopApp),
];
}

TracksCompanion _trackRow(Track track) => TracksCompanion.insert(
bvid: track.bvid,
cid: track.cid,
title: track.title,
cleanedName: Value(track.cleanedName),
artist: Value(track.artist),
duration: track.duration.inSeconds,
cover: Value(absoluteCoverUrl(track.cover)),
uploader: Value(track.uploader),
pageIndex: Value(track.pageIndex),
totalPages: Value(track.totalPages),
);

// ===== lib/app/motion/motion.dart =====
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/scheduler.dart';
import 'package:go_router/go_router.dart';

/// 页面切换、状态替换和列表入场共用的时长。
///
/// 系统开启「减少动态效果」时，过渡时长为零，列表不再错峰入场。
abstract final class Motion {
static const swap = Duration(milliseconds: 260);
static const page = Duration(milliseconds: 320);
```

```
static const pageReverse = Duration(milliseconds: 240);
static const item = Duration(milliseconds: 280);
static const stagger = Duration(milliseconds: 32);

static bool reduced(BuildContext context) =>
MediaQuery.disableAnimationsOf(context);
}

/// 底栏三个分页的切换方向。新页从这一侧进入，旧页向反方向让出。
final tabMotion = TabMotion();

final class TabMotion {
int index = 0;
double travel = 0.045;
bool primed = false;

void reset() {
index = 0;
travel = 0.045;
primed = false;
}

void consume(int next) {
if (next == index) return;
travel = next >= index ? 0.045 : -0.045;
index = next;
}
}

/// 壳层分页。第一次出现不播动画，避免启动时闪一下。
Page<void> shellTabPage({
required BuildContext context,
required LocalKey pageKey,
required String path,
required int tabIndex,
required Widget child,
}) {
final opening = !tabMotion.primed;
tabMotion.primed = true;
tabMotion.consume(tabIndex);
final reduced = Motion.reduced(context);
return CustomTransitionPage<void>(
key: pageKey,
name: path,
child: child,
transitionDuration: opening || reduced ? Duration.zero : Motion.page,
reverseTransitionDuration: reduced ? Duration.zero : Motion.pageReverse,
transitionsBuilder: (context, animation, secondary, child) {
return TabTransition(animation: animation, child: child);
},
```

```
);
}

/// 全屏页。播放页自下而上，其余页横向淡入。
///
/// iOS / macOS 的横向页用系统滑动，保留边缘返回。
Page<void> pushMotionPage({
required BuildContext context,
required LocalKey pageKey,
required String name,
required Widget child,
bool vertical = false,
}) {
if (Motion.reduced(context)) {
return NoTransitionPage<void>(key: pageKey, name: name, child: child);
}
final platform = Theme.of(context).platform;
if (!vertical &&
(platform == TargetPlatform.iOS || platform == TargetPlatform.macOS)) {
return CupertinoPage<void>(key: pageKey, name: name, child: child);
}
final begin = vertical ? const Offset(0, 0.08) : const Offset(0.06, 0);
return CustomTransitionPage<void>(
key: pageKey,
name: name,
opaque: false,
fullscreenDialog: vertical,
transitionDuration: Motion.page,
reverseTransitionDuration: Motion.pageReverse,
child: child,
transitionsBuilder: (context, animation, secondary, child) {
return PushTransition(animation: animation, begin: begin, child: child);
},
);
}

/// 底栏切换：旧页先淡出并让开，新页再淡入。中间露出的是壳层底色。
class TabTransition extends StatelessWidget {
const TabTransition({
super.key,
required this.animation,
required this.child,
});

final Animation<double> animation;
final Widget child;

@override
Widget build(BuildContext context) {
return AnimatedBuilder(
```

```
animation: animation,
builder: (context, child) {
final leaving =
animation.status == AnimationStatus.reverse ||
animation.status == AnimationStatus.dismissed;
final travel = tabMotion.travel;
final double opacity;
final double dx;
if (leaving) {
final progress = Curves.easeOut.transform(
_interval(1 - animation.value, 0, 0.35),
);
opacity = 1 - progress;
dx = -travel * progress;
} else {
final progress = Curves.easeOutCubic.transform(
_interval(animation.value, 0.28, 1),
);
opacity = progress;
dx = travel * (1 - progress);
}
return ClipRect(
child: IgnorePointer(
ignoring: opacity < 1,
child: Opacity(
opacity: opacity,
child: FractionalTranslation(
translation: Offset(dx, 0),
child: child,
),
),
),
);
},
child: child,
);
}
}

/// 压入的全屏页：淡入并带一小段位移，返回时沿原路退出。
class PushTransition extends StatelessWidget {
const PushTransition({
super.key,
required this.animation,
required this.begin,
required this.child,
});

final Animation<double> animation;
final Offset begin;
```

```
final Widget child;

@override
Widget build(BuildContext context) {
return AnimatedBuilder(
animation: animation,
builder: (context, child) {
final leaving =
animation.status == AnimationStatus.reverse ||
animation.status == AnimationStatus.dismissed;
final t = (leaving ? Curves.easeInCubic : Curves.easeOutCubic)
.transform(animation.value);
return Opacity(
opacity: t,
child: FractionalTranslation(
translation: Offset(begin.dx * (1 - t), begin.dy * (1 - t)),
child: child,
),
);
},
child: child,
);
}
}

/// 加载、空、出错、有数据之间的淡入替换。
class MotionSwap extends StatelessWidget {
const MotionSwap({super.key, required this.swapKey, required this.child});

final Object swapKey;
final Widget child;

@override
Widget build(BuildContext context) {
final reduced = Motion.reduced(context);
return AnimatedSwitcher(
duration: reduced ? Duration.zero : Motion.swap,
switchInCurve: Curves.easeOutCubic,
switchOutCurve: Curves.easeInCubic,
transitionBuilder: (child, animation) {
return AnimatedBuilder(
animation: animation,
builder: (context, child) {
return Opacity(
opacity: animation.value,
child: Transform.translate(
offset: Offset(0, 8 * (1 - animation.value)),
child: child,
),
);
```

```
},
child: child,
);
},
child: KeyedSubtree(key: ValueKey<Object>(swapKey), child: child),
);
}
}

/// 包住列表。只有出现后很短时间内构建的前几项会入场，
/// 滚出再滚回、或加载更多，都不再重播。
class ListEntranceScope extends StatefulWidget {
const ListEntranceScope({super.key, required this.child});

final Widget child;

@override
State<ListEntranceScope> createState() => _ListEntranceScopeState();
}

class _ListEntranceScopeState extends State<ListEntranceScope> {
late final Duration opened = SchedulerBinding.instance.currentFrameTimeStamp;

@override
Widget build(BuildContext context) {
return _EntranceClock(opened: opened, child: widget.child);
}
}

class _EntranceClock extends InheritedWidget {
const _EntranceClock({required this.opened, required super.child});

final Duration opened;

static Duration? maybeOf(BuildContext context) {
return context.dependOnInheritedWidgetOfExactType<_EntranceClock>()?.opened;
}

@override
bool updateShouldNotify(_EntranceClock oldWidget) =>
opened != oldWidget.opened;
}

class ListEntrance extends StatefulWidget {
const ListEntrance({super.key, required this.index, required this.child});

static const animatedCount = 8;
static const window = Duration(milliseconds: 500);

final int index;
```

```
final Widget child;

@override
State<ListEntrance> createState() => _ListEntranceState();
}

class _ListEntranceState extends State<ListEntrance>
with SingleTickerProviderStateMixin {
late final AnimationController _controller = AnimationController(
vsync: this,
duration: Motion.item,
);
var _delay = 0.0;
var _decided = false;

@override
void didChangeDependencies() {
super.didChangeDependencies();
if (_decided) return;
_decided = true;
final opened = _EntranceClock.maybeOf(context);
final fresh =
opened != null &&
SchedulerBinding.instance.currentFrameTimeStamp - opened <
ListEntrance.window;
final animate =
fresh &&
!Motion.reduced(context) &&
widget.index >= 0 &&
widget.index < ListEntrance.animatedCount;
if (!animate) {
_controller.value = 1;
return;
}
final delay = Motion.stagger * widget.index;
final total = Motion.item + delay;
_delay = delay.inMilliseconds / total.inMilliseconds;
_controller.duration = total;
_controller.forward();
}

@override
void dispose() {
_controller.dispose();
super.dispose();
}

@override
Widget build(BuildContext context) {
if (_controller.value == 1 && !_controller.isAnimating) return widget.child;
```

```
return AnimatedBuilder(
animation: _controller,
builder: (context, child) {
final t = Curves.easeOutCubic.transform(
_interval(_controller.value, _delay, 1),
);
return Opacity(
opacity: t,
child: FractionalTranslation(
translation: Offset(0, 0.22 * (1 - t)),
child: child,
),
);
},
child: widget.child,
);
}
}

/// 列表加载时的扫光骨架。
class SkeletonList extends StatelessWidget {
const SkeletonList({super.key, this.count = 8, this.leading = 56});

final int count;
final double leading;

@override
Widget build(BuildContext context) {
final color = Theme.of(context).colorScheme.surfaceContainerHighest;
Widget bar(double width) => Container(
width: width,
height: 12,
margin: const EdgeInsets.only(bottom: 6),
decoration: BoxDecoration(
color: color,
borderRadius: BorderRadius.circular(4),
),
);
return Shimmer(
child: ListView.builder(
physics: const NeverScrollableScrollPhysics(),
itemCount: count,
itemBuilder: (_, i) => ListTile(
leading: Container(width: leading, height: leading, color: color),
title: bar(i.isEven ? 180 : 140),
subtitle: bar(100),
),
),
);
}
```

```
}

class Shimmer extends StatefulWidget {
const Shimmer({super.key, required this.child});

final Widget child;

@override
State<Shimmer> createState() => _ShimmerState();
}

class _ShimmerState extends State<Shimmer> with SingleTickerProviderStateMixin {
late final AnimationController _controller = AnimationController(
vsync: this,
duration: const Duration(milliseconds: 1400),
);

@override
void didChangeDependencies() {
super.didChangeDependencies();
if (Motion.reduced(context)) {
_controller.stop();
} else if (!_controller.isAnimating) {
_controller.repeat();
}
}

@override
void dispose() {
_controller.dispose();
super.dispose();
}

@override
Widget build(BuildContext context) {
if (Motion.reduced(context)) return widget.child;
final scheme = Theme.of(context).colorScheme;
final base = scheme.surfaceContainerHighest;
final highlight = Color.lerp(base, scheme.onSurface, 0.08) ?? base;
return AnimatedBuilder(
animation: _controller,
builder: (context, child) {
final t = _controller.value;
return ShaderMask(
blendMode: BlendMode.srcATop,
shaderCallback: (bounds) {
return LinearGradient(
begin: Alignment(-1.2 + 2.4 * t, -0.2),
end: Alignment(-0.2 + 2.4 * t, 0.2),
colors: [base, highlight, base],
```

```
stops: const [0.25, 0.5, 0.75],
).createShader(bounds);
},
child: child,
);
},
child: widget.child,
);
}
}

double _interval(double t, double start, double end) {
if (t <= start) return 0;
if (t >= end) return 1;
return (t - start) / (end - start);
}

// ===== lib/app/platform/app_quit.dart =====
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:tray_manager/tray_manager.dart';

import '../../core/logging/app_logger.dart';
import 'window_frame.dart';

/// 保存窗口位置，去掉托盘图标，然后结束桌面端进程。
///
/// 手机端直接返回。托盘还没建好时仍会退出。
Future<void> quitDesktopApp() async {
if (kIsWeb) return;
if (!(Platform.isMacOS || Platform.isWindows || Platform.isLinux)) return;
try {
await WindowFrameStore.instance?.persist();
} catch (error, stack) {
AppLogger.e(
'save window before quit failed',
tag: 'update',
error: error,
stackTrace: stack,
);
}
try {
await trayManager.destroy();
} catch (error, stack) {
AppLogger.e(
'tray destroy before quit failed',
tag: 'update',
error: error,
stackTrace: stack,
```

```
padding: EdgeInsets.all(16),
child: Center(child: CircularProgressIndicator()),
);
} else if (state.error != null) {
key = 'error';
body = TextButton(
onPressed: onRetry,
child: Text(errorText(context.l10n, state.error)),
);
} else if (onLoadMore != null && state.hasMore) {
key = 'more';
body = TextButton(
onPressed: onLoadMore,
child: Text(context.l10n.searchLoadMore),
);
} else {
key = 'end';
body = const SizedBox(height: 16);
}
return MotionSwap(swapKey: key, child: body);
}
}

Object _searchBodyKey(SearchState state, List<SearchHit> items) {
if (state.items.isEmpty && state.loading) return 'loading';
if (state.items.isEmpty && state.error != null) return 'error';
if (items.isEmpty &&
!state.loading &&
state.error == null &&
!state.hasMore) {
return state.keyword.isEmpty ? 'daily-empty' : 'empty:${state.keyword}';
}
return state.keyword.isEmpty
? 'daily:${state.chart.name}'
: 'results:${state.keyword}';
}

String _discoverGroupLabel(AppLocalizations l10n, DiscoverGroup group) =>
switch (group) {
DiscoverGroup.ranking => l10n.discoverGroupRanking,
DiscoverGroup.era => l10n.discoverGroupEra,
};

String _discoverTabLabel(AppLocalizations l10n, DiscoverTab tab) =>
switch (tab) {
DiscoverTab.heat => l10n.discoverTabHeat,
DiscoverTab.rating => l10n.discoverTabRating,
DiscoverTab.original => l10n.discoverTabOriginal,
DiscoverTab.cover => l10n.discoverTabCover,
DiscoverTab.mv => l10n.discoverTabMv,
```

```
DiscoverTab.live => l10n.discoverTabLive,
DiscoverTab.perform => l10n.discoverTabPerform,
DiscoverTab.vocaloid => l10n.discoverTabVocaloid,
DiscoverTab.ai => l10n.discoverTabAi,
DiscoverTab.y2020 => l10n.discoverTabY2020,
DiscoverTab.y2010 => l10n.discoverTabY2010,
DiscoverTab.y2000 => l10n.discoverTabY2000,
DiscoverTab.y1990 => l10n.discoverTabY1990,
DiscoverTab.y1980 => l10n.discoverTabY1980,
DiscoverTab.y1970 => l10n.discoverTabY1970,
};

class _ChartTabs extends StatelessWidget {
const _ChartTabs({
required this.selected,
required this.showPlay,
required this.count,
required this.onSelected,
required this.onSelectGroup,
required this.onPlayAll,
});

final DiscoverTab selected;
final bool showPlay;
final int count;
final ValueChanged<DiscoverTab> onSelected;
final ValueChanged<DiscoverGroup> onSelectGroup;
final VoidCallback onPlayAll;

@override
Widget build(BuildContext context) {
final l10n = context.l10n;
final tabs = selected.group == DiscoverGroup.ranking
? DiscoverTab.ranking
: DiscoverTab.eras;
return Padding(
padding: const EdgeInsets.fromLTRB(16, 0, 16, 4),
child: Column(
crossAxisAlignment: CrossAxisAlignment.stretch,
children: [
Row(
children: [
for (final group in DiscoverGroup.values) ...[
ChoiceChip(
label: Text(_discoverGroupLabel(l10n, group)),
selected: group == selected.group,
onSelected: (_) => onSelectGroup(group),
),
const SizedBox(width: 8),
],
```

```
],
),
const SizedBox(height: 4),
_FoldableChips(
key: ValueKey(selected.group),
selectedIndex: tabs.indexOf(selected),
children: [
for (final tab in tabs)
ChoiceChip(
label: Text(_discoverTabLabel(l10n, tab)),
selected: tab == selected,
onSelected: (_) => onSelected(tab),
),
],
),
// 播放全部独占一行，避免和分类抢同一行的宽度。
if (showPlay) ...[
const SizedBox(height: 8),
Row(
children: [
FilledButton.icon(
style: FilledButton.styleFrom(
visualDensity: VisualDensity.compact,
tapTargetSize: MaterialTapTargetSize.shrinkWrap,
padding: const EdgeInsets.symmetric(horizontal: 16),
),
icon: const Icon(Icons.play_arrow),
label: Text(l10n.playlistPlayAll),
onPressed: onPlayAll,
),
const Spacer(),
Text(l10n.playlistTrackCount(count)),
],
),
],
],
),
);
}
}

/// 分类芯片按可用宽度平铺。一行放得下就全部显示；放不下时先收成一行，
/// 末尾给出展开，展开后再按行铺开。
class _FoldableChips extends StatefulWidget {
const _FoldableChips({
super.key,
required this.selectedIndex,
required this.children,
});
```

```
final int selectedIndex;
final List<Widget> children;

@override
State<_FoldableChips> createState() => _FoldableChipsState();
}

class _FoldableChipsState extends State<_FoldableChips> {
bool _expanded = false;

@override
Widget build(BuildContext context) {
final material = MaterialLocalizations.of(context);
return AnimatedSize(
alignment: Alignment.topCenter,
duration: Motion.reduced(context) ? Duration.zero : Motion.swap,
curve: Curves.easeOutCubic,
child: _ChipFlow(
expanded: _expanded,
selectedIndex: widget.selectedIndex,
spacing: 8,
runSpacing: 4,
children: [
...widget.children,
IconButton(
tooltip: _expanded
? material.expandedIconTapHint
: material.collapsedIconTapHint,
style: IconButton.styleFrom(
tapTargetSize: MaterialTapTargetSize.shrinkWrap,
minimumSize: const Size(36, 36),
fixedSize: const Size(36, 36),
padding: EdgeInsets.zero,
iconSize: 22,
),
onPressed: () => setState(() => _expanded = !_expanded),
icon: Icon(_expanded ? Icons.expand_less : Icons.expand_more),
),
],
),
);
}
}

class _ChipFlow extends MultiChildRenderObjectWidget {
const _ChipFlow({
required this.expanded,
required this.selectedIndex,
required this.spacing,
required this.runSpacing,
```

```
required super.children,
});

final bool expanded;
final int selectedIndex;
final double spacing;
final double runSpacing;

@override
MultiChildRenderObjectElement createElement() => _ChipFlowElement(this);

@override
RenderObject createRenderObject(BuildContext context) {
return _RenderChipFlow(
expanded: expanded,
selectedIndex: selectedIndex,
spacing: spacing,
runSpacing: runSpacing,
);
}

@override
void updateRenderObject(BuildContext context, _RenderChipFlow renderObject) {
renderObject
..expanded = expanded
..selectedIndex = selectedIndex
..spacing = spacing
..runSpacing = runSpacing;
}
}

class _ChipFlowElement extends MultiChildRenderObjectElement {
_ChipFlowElement(super.widget);

@override
void debugVisitOnstageChildren(ElementVisitor visitor) {
final flow = renderObject as _RenderChipFlow;
for (final child in children) {
final object = child.renderObject;
if (object is RenderBox && flow.hides(object)) continue;
visitor(child);
}
}
}

class _ChipFlowParentData extends ContainerBoxParentData<RenderBox> {}

class _RenderChipFlow extends RenderBox
with
ContainerRenderObjectMixin<RenderBox, _ChipFlowParentData>,
```

```
RenderBoxContainerDefaultsMixin<RenderBox, _ChipFlowParentData> {
_RenderChipFlow({
required bool expanded,
required int selectedIndex,
required double spacing,
required double runSpacing,
}) : _expanded = expanded,
_selectedIndex = selectedIndex,
_spacing = spacing,
_runSpacing = runSpacing;

bool _expanded;
int _selectedIndex;
double _spacing;
double _runSpacing;
final Set<RenderBox> _hidden = <RenderBox>{};

bool hides(RenderBox child) => _hidden.contains(child);

set expanded(bool value) {
if (_expanded == value) return;
_expanded = value;
markNeedsLayout();
}

set selectedIndex(int value) {
if (_selectedIndex == value) return;
_selectedIndex = value;
markNeedsLayout();
}

set spacing(double value) {
if (_spacing == value) return;
_spacing = value;
markNeedsLayout();
}

set runSpacing(double value) {
if (_runSpacing == value) return;
_runSpacing = value;
markNeedsLayout();
}

@override
void setupParentData(RenderBox child) {
if (child.parentData is! _ChipFlowParentData) {
child.parentData = _ChipFlowParentData();
}
}
```

```
List<RenderBox> get _boxes {
final list = <RenderBox>[];
var child = firstChild;
while (child != null) {
list.add(child);
child = childAfter(child);
}
return list;
}

double _rowWidth(List<RenderBox> row) {
var width = 0.0;
for (var i = 0; i < row.length; i++) {
if (i > 0) width += _spacing;
width += row[i].size.width;
}
return width;
}

void _place(RenderBox child, Offset offset) {
final data = child.parentData! as _ChipFlowParentData;
data.offset = offset;
}

double _placeRow(List<RenderBox> row, double y) {
var rowHeight = 0.0;
for (final child in row) {
rowHeight = math.max(rowHeight, child.size.height);
}
var x = 0.0;
for (final child in row) {
_place(child, Offset(x, y + (rowHeight - child.size.height) / 2));
x += child.size.width + _spacing;
}
return rowHeight;
}

@override
void performLayout() {
final boxes = _boxes;
final maxWidth = constraints.maxWidth;
final childConstraints = maxWidth.isFinite
? BoxConstraints(maxWidth: maxWidth)
: const BoxConstraints();
for (final child in boxes) {
child.layout(childConstraints, parentUsesSize: true);
}
_hidden.clear();
if (boxes.isEmpty) {
size = constraints.smallest;
```

```
return;
}

final toggle = boxes.last;
final chips = boxes.sublist(0, boxes.length - 1);
final widthLimit = maxWidth.isFinite ? maxWidth : double.infinity;

var oneLine = 0.0;
var fits = true;
for (var i = 0; i < chips.length; i++) {
final next = i == 0
? chips[i].size.width
: oneLine + _spacing + chips[i].size.width;
if (next > widthLimit) {
fits = false;
break;
}
oneLine = next;
}

if (chips.isEmpty || fits) {
_hidden.add(toggle);
final height = chips.isEmpty ? 0.0 : _placeRow(chips, 0);
for (final child in _hidden) {
_place(child, Offset.zero);
}
final width = maxWidth.isFinite ? maxWidth : oneLine;
size = constraints.constrain(Size(width, height));
markNeedsSemanticsUpdate();
return;
}

if (!_expanded) {
final room = math.max(0.0, widthLimit - _spacing - toggle.size.width);
final visible = <RenderBox>[];
var used = 0.0;
for (final chip in chips) {
final next = visible.isEmpty
? chip.size.width
: used + _spacing + chip.size.width;
if (visible.isNotEmpty && next > room) break;
visible.add(chip);
used = next;
}
final selected = _selectedIndex;
if (selected >= 0 && selected < chips.length) {
final chip = chips[selected];
if (!visible.contains(chip)) {
while (visible.isNotEmpty &&
_rowWidth(visible) + _spacing + chip.size.width > room) {
```

```
visible.removeLast();
}
visible.add(chip);
}
}
for (final chip in chips) {
if (!visible.contains(chip)) _hidden.add(chip);
}
final row = <RenderBox>[...visible, toggle];
final height = _placeRow(row, 0);
final toggleOffset = toggle.parentData! as _ChipFlowParentData;
final pinned = math.min(
toggleOffset.offset.dx,
math.max(0.0, widthLimit - toggle.size.width),
);
_place(toggle, Offset(pinned, toggleOffset.offset.dy));
for (final child in _hidden) {
_place(child, Offset.zero);
}
size = constraints.constrain(Size(widthLimit, height));
markNeedsSemanticsUpdate();
return;
}

final rows = <List<RenderBox>>[<RenderBox>[]];
var used = 0.0;
void add(RenderBox item) {
final itemWidth = item.size.width;
if (rows.last.isNotEmpty && used + _spacing + itemWidth > widthLimit) {
rows.add(<RenderBox>[]);
used = 0;
}
final row = rows.last;
used = row.isEmpty ? itemWidth : used + _spacing + itemWidth;
row.add(item);
}

for (final chip in chips) {
add(chip);
}
add(toggle);
var y = 0.0;
for (var i = 0; i < rows.length; i++) {
if (rows[i].isEmpty) continue;
y += _placeRow(rows[i], y);
if (i != rows.length - 1) y += _runSpacing;
}
size = constraints.constrain(Size(widthLimit, y));
markNeedsSemanticsUpdate();
}
```

```
@override
void paint(PaintingContext context, Offset offset) {
context.pushClipRect(needsCompositing, offset, Offset.zero & size, (
context,
offset,
) {
var child = firstChild;
while (child != null) {
if (!_hidden.contains(child)) {
final data = child.parentData! as _ChipFlowParentData;
context.paintChild(child, offset + data.offset);
}
child = childAfter(child);
}
});
}

@override
bool hitTestChildren(BoxHitTestResult result, {required Offset position}) {
var child = lastChild;
while (child != null) {
final current = child;
if (!_hidden.contains(current)) {
final data = current.parentData! as _ChipFlowParentData;
final hit = result.addWithPaintOffset(
offset: data.offset,
position: position,
hitTest: (result, transformed) {
return current.hitTest(result, position: transformed);
},
);
if (hit) return true;
}
child = childBefore(current);
}
return false;
}

@override
void visitChildrenForSemantics(RenderObjectVisitor visitor) {
var child = firstChild;
while (child != null) {
if (!_hidden.contains(child)) visitor(child);
child = childAfter(child);
}
}
}

// ===== lib/features/settings/cache_page.dart =====
```

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../app/adaptive/max_width_box.dart';
import '../../app/motion/motion.dart';
import '../../app/providers.dart';
import '../../l10n/l10n.dart';

final cacheStatsProvider = FutureProvider.autoDispose<(int, int)>(
(ref) => ref.watch(audioHandlerProvider).downloads.store.stats(),
);

/// 缓存管理占位 (M6.7, 完整界面见 v0.2.3)
class CachePage extends StatelessWidget {
const CachePage({super.key});

@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: Text(context.l10n.cacheTitle)),
body: const MaxWidthBox(maxWidth: 640, child: _CacheBody()),
);
}
}

class _CacheBody extends ConsumerWidget {
const _CacheBody();

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final stats = ref.watch(cacheStatsProvider);

Future<void> clear() async {
final ok = await showDialog<bool>(
context: context,
builder: (c) => AlertDialog(
content: Text(l10n.cacheClearConfirm),
actions: [
TextButton(
onPressed: () => Navigator.pop(c, false),
child: Text(l10n.cancel),
),
FilledButton(
onPressed: () => Navigator.pop(c, true),
child: Text(l10n.confirm),
),
],
),
);
```

```
if (ok != true) return;
final downloads = ref.read(audioHandlerProvider).downloads;
final n = await downloads.store.clearUnpinned(
protectedKeys: downloads.activeKeys,
);
ref.invalidate(cacheStatsProvider);
if (context.mounted) {
ScaffoldMessenger.of(context)
.showSnackBar(SnackBar(content: Text(l10n.cacheCleared(n))));
}
}

return Center(
child: MotionSwap(
swapKey: stats.when(
loading: () => 'loading',
error: (_, _) => 'error',
data: (_) => 'ready',
),
child: stats.when(
loading: () => const CircularProgressIndicator(),
error: (e, _) => Text(errorText(l10n, e)),
data: (s) => Column(
mainAxisSize: MainAxisSize.min,
children: [
Text(
l10n.cacheSummary(s.$1, formatBytes(s.$2)),
style: Theme.of(context).textTheme.titleMedium,
),
const SizedBox(height: 20),
FilledButton.tonal(
onPressed: s.$1 == 0 && s.$2 == 0 ? null : clear,
child: Text(l10n.cacheClearAll),
),
],
),
),
),
);
}
}

// ===== lib/features/settings/settings_page.dart =====
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../app/adaptive/layout_settings.dart';
import '../../app/adaptive/max_width_box.dart';
import '../../app/platform/open_url.dart';
```

```
import '../../app/providers.dart';
import '../../app/theme.dart';
import '../../domain/entities/audio_track.dart';
import '../../domain/update/distribution.dart';
import '../../l10n/l10n.dart';
import '../common/cover_image.dart';
import '../login/login_controller.dart';
import 'update_prompt.dart';

/// 设置 (M6.6)
class SettingsPage extends ConsumerWidget {
const SettingsPage({super.key});

@override
Widget build(BuildContext context, WidgetRef ref) {
return const MaxWidthBox(child: _SettingsList());
}
}

class _SettingsList extends ConsumerWidget {
const _SettingsList();

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final db = ref.watch(databaseProvider);
final prefs = ref.watch(playbackPrefsProvider);
return ListView(
padding: const EdgeInsets.fromLTRB(16, 16, 16, 24),
children: [
const _SettingsGroup(children: [_AccountTile()]),
_SettingsGroup(
children: [
ListTile(
leading: const Icon(Icons.music_note_outlined),
title: Text(l10n.settingsPlayback),
),
ListTile(
title: Text(l10n.settingsQuality),
subtitle: Text(prefs.quality.name),
onTap: () async {
final q = await _pickQuality(context, prefs.quality);
if (q != null) {
prefs.quality = q;
await db.settingsDao.setDefaultQuality(q.id);
}
},
),
if (!prefs.loggedIn)
Padding(
```

```
padding: const EdgeInsets.fromLTRB(72, 0, 16, 8),
child: Text(
l10n.settingsQualityGuestHint,
style: Theme.of(context).textTheme.bodySmall,
),
),
],
),
_SettingsGroup(
children: [
ListTile(
leading: const Icon(Icons.folder_outlined),
title: Text(l10n.settingsCache),
),
FutureBuilder<int>(
future: db.settingsDao.getCacheMaxSize(),
builder: (c, snap) {
final max = snap.data ?? (500 * 1024 * 1024);
return ListTile(
title: Text(l10n.settingsCacheLimit),
subtitle: Text(formatBytes(max)),
onTap: () => _pickCacheLimit(c, ref, max),
);
},
),
ListTile(
title: Text(l10n.settingsCacheManage),
trailing: const Icon(Icons.chevron_right),
onTap: () => context.push('/cache'),
),
],
),
_SettingsGroup(
children: [
ListTile(
leading: const Icon(Icons.palette_outlined),
title: Text(l10n.settingsAppearance),
),
const _ThemeModeRow(),
const _CarLayoutRow(),
const _SeedColorRow(),
],
),
if (offersInAppUpdate())
const _SettingsGroup(children: [UpdateSettingsTile()]),
_SettingsGroup(
children: [
ListTile(
leading: const Icon(Icons.privacy_tip_outlined),
title: Text(l10n.settingsPrivacy),
```

```
onTap: () => openPrivacyDocument(context, privacyPolicyUrl),
),
ListTile(
leading: const Icon(Icons.child_care_outlined),
title: Text(l10n.settingsChildrenPrivacy),
onTap: () =>
openPrivacyDocument(context, childrenPrivacyPolicyUrl),
),
],
),
],
);
}

Future<AudioQuality?> _pickQuality(
BuildContext context,
AudioQuality current,
) => showDialog<AudioQuality>(
context: context,
builder: (c) => AlertDialog(
title: Text(context.l10n.settingsQuality),
content: RadioGroup<AudioQuality>(
groupValue: current,
onChanged: (v) => Navigator.pop(c, v),
child: Column(
mainAxisSize: MainAxisSize.min,
children: [
for (final q in AudioQuality.values.reversed)
RadioListTile<AudioQuality>(title: Text(q.name), value: q),
],
),
),
),
);

Future<void> _pickCacheLimit(
BuildContext context,
WidgetRef ref,
int current,
) async {
final l10n = context.l10n;
final options = [
250 * 1024 * 1024,
500 * 1024 * 1024,
1024 * 1024 * 1024,
2 * 1024 * 1024 * 1024,
5 * 1024 * 1024 * 1024,
];
final picked = await showDialog<int>(
context: context,
```

```
builder: (c) => AlertDialog(
title: Text(l10n.settingsCacheLimit),
content: RadioGroup<int>(
groupValue: current,
onChanged: (v) => Navigator.pop(c, v),
child: Column(
mainAxisSize: MainAxisSize.min,
children: [
for (final size in options)
RadioListTile<int>(title: Text(formatBytes(size)), value: size),
],
),
),
),
);
if (picked != null) {
await ref.read(databaseProvider).settingsDao.setCacheMaxSize(picked);
}
}
}

class _SettingsGroup extends StatelessWidget {
const _SettingsGroup({required this.children});

final List<Widget> children;

@override
Widget build(BuildContext context) {
final scheme = Theme.of(context).colorScheme;
return Padding(
padding: const EdgeInsets.only(bottom: 12),
child: Material(
color: scheme.surfaceContainerLow,
surfaceTintColor: Colors.transparent,
elevation: 0,
borderRadius: BorderRadius.circular(16),
clipBehavior: Clip.antiAlias,
child: Column(mainAxisSize: MainAxisSize.min, children: children),
),
);
}
}

class _ThemeModeRow extends ConsumerWidget {
const _ThemeModeRow();

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final controller = ref.watch(themeControllerProvider.notifier);
```

```
final mode = ref.watch(themeControllerProvider).mode;
return ListTile(
title: Text(l10n.settingsThemeMode),
subtitle: Text(switch (mode) {
ThemeMode.system => l10n.themeSystem,
ThemeMode.light => l10n.themeLight,
ThemeMode.dark => l10n.themeDark,
}),
onTap: () async {
final picked = await showDialog<ThemeMode>(
context: context,
builder: (c) => AlertDialog(
title: Text(l10n.settingsThemeMode),
content: RadioGroup<ThemeMode>(
groupValue: mode,
onChanged: (v) => Navigator.pop(c, v),
child: Column(
mainAxisSize: MainAxisSize.min,
children: [
for (final m in ThemeMode.values)
RadioListTile<ThemeMode>(
title: Text(switch (m) {
ThemeMode.system => l10n.themeSystem,
ThemeMode.light => l10n.themeLight,
ThemeMode.dark => l10n.themeDark,
}),
value: m,
),
],
),
),
),
);
if (picked != null) controller.setMode(picked);
},
);
}
}

class _CarLayoutRow extends ConsumerWidget {
const _CarLayoutRow();

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final mode = ref.watch(layoutSettingsProvider).carMode;
final label = switch (mode) {
CarLayoutMode.auto => l10n.carLayoutAuto,
CarLayoutMode.on => l10n.carLayoutOn,
CarLayoutMode.off => l10n.carLayoutOff,
```

```
};
return ListTile(
title: Text(l10n.settingsCarLayout),
subtitle: Text(label),
onTap: () async {
final picked = await showDialog<CarLayoutMode>(
context: context,
builder: (c) => AlertDialog(
title: Text(l10n.settingsCarLayout),
content: RadioGroup<CarLayoutMode>(
groupValue: mode,
onChanged: (v) => Navigator.pop(c, v),
child: Column(
mainAxisSize: MainAxisSize.min,
children: [
for (final value in CarLayoutMode.values)
RadioListTile<CarLayoutMode>(
title: Text(switch (value) {
CarLayoutMode.auto => l10n.carLayoutAuto,
CarLayoutMode.on => l10n.carLayoutOn,
CarLayoutMode.off => l10n.carLayoutOff,
}),
value: value,
),
],
),
),
),
);
if (picked != null) {
await ref.read(layoutSettingsProvider.notifier).setCarMode(picked);
}
},
);
}
}

class _SeedColorRow extends ConsumerWidget {
const _SeedColorRow();

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final controller = ref.watch(themeControllerProvider.notifier);
final seed = ref.watch(themeControllerProvider).seed;
return ListTile(
title: Text(l10n.settingsSeedColor),
subtitle: Wrap(
spacing: 8,
children: [
```

```
for (final c in presetSeeds)
GestureDetector(
onTap: () => controller.setSeed(c),
child: Container(
width: 36,
height: 36,
decoration: BoxDecoration(
color: c,
shape: BoxShape.circle,
border: seed == c ? Border.all(width: 3, color: c) : null,
),
),
),
],
),
);
}
}

/// 账号 (M7.2): 未登录显示"点击登录", 已登录显示头像/昵称/退出
class _AccountTile extends ConsumerWidget {
const _AccountTile();

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final user = ref.watch(loginControllerProvider);
final u = user.value;
if (u == null) {
return ListTile(
leading: const Icon(Icons.account_circle_outlined),
title: Text(l10n.settingsAccount),
subtitle: Text(user.isLoading ? '…' : l10n.settingsNotLoggedIn),
trailing: const Icon(Icons.chevron_right),
onTap: () => context.push('/login'),
);
}
return ListTile(
leading: CoverImage(u.face, size: 40, radius: 20),
title: Text(u.name),
subtitle: Text('Lv${u.level}${u.vip ? ' · VIP' : ''}'),
trailing: TextButton(
child: Text(l10n.logout),
onPressed: () async {
final ok = await showDialog<bool>(
context: context,
builder: (c) => AlertDialog(
content: Text(l10n.logoutConfirm),
actions: [
TextButton(
```

```
onPressed: () => Navigator.pop(c, false),
child: Text(l10n.cancel),
),
FilledButton(
onPressed: () => Navigator.pop(c, true),
child: Text(l10n.confirm),
),
],
),
);
if (ok == true) {
await ref.read(loginControllerProvider.notifier).logout();
}
},
),
);
}
}

// ===== lib/features/settings/update_controller.dart =====
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/logging/app_logger.dart';
import '../../data/update/app_release.dart';
import '../../data/update/app_version_source.dart';
import '../../data/update/host_platform.dart';
import '../../data/update/local_update_installer.dart';
import '../../data/update/release_source.dart';
import '../../domain/update/app_version.dart';
import '../../domain/update/distribution.dart';
import '../../domain/update/installer_asset.dart';

/// 一次升级检查或安装所处的阶段。
enum UpdatePhase {
/// 还没检查。
idle,

/// 正在请求 GitHub。
checking,

/// 没有更新的正式版。
upToDate,

/// 有更新，并且 Release 里有本机的安装包。
available,

/// 有更新，但没有本机可安装的附件。
unsupported,

/// 正在下载安装包。
```

```
downloading,

/// 已经交给系统打开安装包。
launched,

/// 检查或下载失败。
failed,
}

/// 失败发生在哪一步。
enum UpdateFailure {
/// 检查更新的请求失败。
check,

/// 下载或打开安装包失败。
download,

/// Android 尚未允许安装未知应用。
permission,
}

/// 升级界面使用的状态。
class UpdateState {
/// 创建状态。
const UpdateState({
this.phase = UpdatePhase.idle,
this.current,
this.release,
this.asset,
this.progress = 0,
this.failure,
});

/// 当前阶段。
final UpdatePhase phase;

/// 本机版本。还没读到时为 `null`。
final AppVersion? current;

/// 远程正式版。没有正式版时为 `null`。
final AppRelease? release;

/// 准备下载的附件。
final ReleaseAsset? asset;

/// 下载进度，0 到 1。
final double progress;

/// 最近一次失败。没有失败时为 `null`。
final UpdateFailure? failure;
```

```
/// 复制并替换给出的字段。[failure] 传入 `null` 会清掉失败。
UpdateState copyWith({
UpdatePhase? phase,
AppVersion? current,
AppRelease? release,
ReleaseAsset? asset,
double? progress,
Object? failure = _keepFailure,
}) => UpdateState(
phase: phase ?? this.phase,
current: current ?? this.current,
release: release ?? this.release,
asset: asset ?? this.asset,
progress: progress ?? this.progress,
failure: identical(failure, _keepFailure)
? this.failure
: failure as UpdateFailure?,
);
}

const Object _keepFailure = Object();

/// 当前版本来源。正式启动时由 bootstrap 换成包信息。
final appVersionSourceProvider = Provider<AppVersionSource>(
(ref) => FixedAppVersionSource(AppVersion.parse('0.1.0+1')!),
);

/// 升级源。正式启动时换成 GitHub Releases。
final releaseSourceProvider = Provider<ReleaseSource>(
(ref) => const EmptyReleaseSource(),
);

/// 本机系统与 CPU。测试可覆盖，避免依赖运行测试的机器。
final updateHostProvider = Provider<UpdateHost>((ref) => currentUpdateHost());

/// 下载并打开安装包。
final updateInstallerProvider = Provider<UpdateInstaller>(
(ref) => const NoopUpdateInstaller(),
);

/// 安装包打开后的退出动作。测试中保持为空操作。
final appQuitProvider = Provider<Future<void> Function()>((ref) => () async {});

/// 检查 GitHub 正式版，并在有安装包时下载打开。
class UpdateController extends Notifier<UpdateState> {
@override
UpdateState build() => const UpdateState();

/// 向升级源查询是否有比本机新的正式版。
```

```
Future<void> check() async {
if (!offersInAppUpdate()) return;
if (state.phase == UpdatePhase.downloading) return;
state = UpdateState(phase: UpdatePhase.checking, current: state.current);
try {
final current = await ref.read(appVersionSourceProvider).current();
final release = await ref.read(releaseSourceProvider).latestStable();
if (release == null || !release.version.isNewerThan(current)) {
state = UpdateState(phase: UpdatePhase.upToDate, current: current);
return;
}
final host = ref.read(updateHostProvider);
final name = installerAssetName(
operatingSystem: host.operatingSystem,
architecture: host.architecture,
);
final asset = name == null ? null : release.assetByName(name);
state = UpdateState(
phase: asset == null ? UpdatePhase.unsupported : UpdatePhase.available,
current: current,
release: release,
asset: asset,
);
} catch (error, stack) {
AppLogger.e(
'update check failed',
tag: 'update',
error: error,
stackTrace: stack,
);
state = UpdateState(
phase: UpdatePhase.failed,
failure: UpdateFailure.check,
current: state.current,
);
}
}

/// 下载当前选中的安装包并打开，然后退出桌面端进程。
Future<void> downloadAndInstall() async {
if (!offersInAppUpdate()) return;
final asset = state.asset;
if (asset == null || state.phase == UpdatePhase.downloading) return;
final current = state.current;
final release = state.release;
state = UpdateState(
phase: UpdatePhase.downloading,
current: current,
release: release,
asset: asset,
```

```
);
try {
await ref
.read(updateInstallerProvider)
.install(
asset,
onProgress: (progress) {
state = state.copyWith(progress: progress);
},
);
state = UpdateState(
phase: UpdatePhase.launched,
current: current,
release: release,
asset: asset,
progress: 1,
);
await ref.read(appQuitProvider)();
} on InstallPermissionException {
state = UpdateState(
phase: UpdatePhase.available,
current: current,
release: release,
asset: asset,
failure: UpdateFailure.permission,
);
} catch (error, stack) {
AppLogger.e(
'update install failed',
tag: 'update',
error: error,
stackTrace: stack,
);
state = UpdateState(
phase: UpdatePhase.available,
current: current,
release: release,
asset: asset,
failure: UpdateFailure.download,
);
}
}
}

/// 升级状态。
final updateControllerProvider =
NotifierProvider<UpdateController, UpdateState>(UpdateController.new);

// ===== lib/features/settings/update_prompt.dart =====
import 'package:flutter/material.dart';
```

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../app/adaptive/layout_settings.dart';
import '../../domain/update/distribution.dart';
import '../../l10n/l10n.dart';
import 'update_controller.dart';

/// 进入主界面后检查一次更新。车机布局下不弹窗。
class UpdatePrompt extends ConsumerStatefulWidget {
/// 包住主界面。
const UpdatePrompt({super.key, required this.child});

/// 被包住的子树。
final Widget child;

@override
ConsumerState<UpdatePrompt> createState() => _UpdatePromptState();
}

class _UpdatePromptState extends ConsumerState<UpdatePrompt> {
var _shown = false;

@override
void initState() {
super.initState();
WidgetsBinding.instance.addPostFrameCallback((_) {
if (!mounted || !offersInAppUpdate()) return;
ref.read(updateControllerProvider.notifier).check();
});
}

void _schedulePrompt() {
WidgetsBinding.instance.addPostFrameCallback((_) => _maybePrompt());
}

void _maybePrompt() {
if (!offersInAppUpdate() || _shown || !mounted) return;
if (ref.read(layoutSettingsProvider).carLayout) return;
final phase = ref.read(updateControllerProvider).phase;
if (phase != UpdatePhase.available && phase != UpdatePhase.unsupported) {
return;
}
_shown = true;
showUpdateDialog(context);
}

@override
Widget build(BuildContext context) {
ref.listen(updateControllerProvider, (previous, next) {
if (previous?.phase == next.phase) return;
```

```
_schedulePrompt();
});
ref.listen(layoutSettingsProvider, (previous, next) => _schedulePrompt());
return widget.child;
}
}

/// 打开升级对话框。
Future<void> showUpdateDialog(BuildContext context) {
if (!offersInAppUpdate()) return Future<void>.value();
return showDialog<void>(
context: context,
builder: (context) => const _UpdateDialog(),
);
}

class _UpdateDialog extends ConsumerWidget {
const _UpdateDialog();

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final state = ref.watch(updateControllerProvider);
final version = state.release?.version.text ?? '';
final notes = state.release?.changelog ?? '';
return AlertDialog(
title: Text(
state.phase == UpdatePhase.unsupported
? l10n.updateUnsupportedTitle
: l10n.updateAvailableTitle(version),
),
content: ConstrainedBox(
constraints: const BoxConstraints(maxWidth: 420),
child: Column(
mainAxisSize: MainAxisSize.min,
crossAxisAlignment: CrossAxisAlignment.start,
children: [
if (notes.isNotEmpty)
ConstrainedBox(
constraints: const BoxConstraints(maxHeight: 220),
child: SingleChildScrollView(child: Text(notes)),
),
if (state.phase == UpdatePhase.unsupported &&
(state.release?.pageUrl.isNotEmpty ?? false)) ...[
const SizedBox(height: 12),
SelectableText(state.release!.pageUrl),
],
if (state.phase == UpdatePhase.downloading) ...[
const SizedBox(height: 16),
LinearProgressIndicator(
```

```
value: state.progress == 0 ? null : state.progress,
),
const SizedBox(height: 8),
Text(l10n.updateDownloading((state.progress * 100).round())),
if (state.asset?.name.endsWith('.dmg') == true) ...[
const SizedBox(height: 8),
Text(l10n.updateMacAdmin),
],
],
if (state.failure != null) ...[
const SizedBox(height: 12),
Text(_failureText(l10n, state.failure!)),
],
if (state.phase == UpdatePhase.launched) ...[
const SizedBox(height: 12),
Text(
state.asset?.name.endsWith('.ipa') == true
? l10n.updateLaunchedIos
: state.asset?.name.endsWith('.dmg') == true
? l10n.updateLaunchedMac
: l10n.updateLaunched,
),
],
],
),
),
actions: [
if (state.phase != UpdatePhase.launched &&
state.phase != UpdatePhase.unsupported)
TextButton(
onPressed: () => Navigator.pop(context),
child: Text(l10n.updateLater),
),
if (state.phase == UpdatePhase.available ||
state.failure == UpdateFailure.permission ||
state.failure == UpdateFailure.download)
FilledButton(
onPressed: state.phase == UpdatePhase.downloading
? null
: () => ref
.read(updateControllerProvider.notifier)
.downloadAndInstall(),
child: Text(
state.asset?.name.endsWith('.ipa') == true
? l10n.updateInstallIos
: l10n.updateDownload,
),
),
if (state.phase == UpdatePhase.launched ||
state.phase == UpdatePhase.unsupported)
```

```
FilledButton(
onPressed: () => Navigator.pop(context),
child: Text(l10n.confirm),
),
],
);
}
}

String _failureText(AppLocalizations l10n, UpdateFailure failure) =>
switch (failure) {
UpdateFailure.check => l10n.updateCheckFailed,
UpdateFailure.download => l10n.updateDownloadFailed,
UpdateFailure.permission => l10n.updatePermission,
};

/// 设置页里的升级入口。
class UpdateSettingsTile extends ConsumerWidget {
/// 创建升级入口。
const UpdateSettingsTile({super.key});

@override
Widget build(BuildContext context, WidgetRef ref) {
final l10n = context.l10n;
final state = ref.watch(updateControllerProvider);
return ListTile(
leading: const Icon(Icons.system_update_outlined),
title: Text(l10n.updateTitle),
subtitle: Text(_subtitle(l10n, state)),
onTap:
state.phase == UpdatePhase.checking ||
state.phase == UpdatePhase.downloading
? null
: () => _onTap(context, ref, state),
);
}

Future<void> _onTap(
BuildContext context,
WidgetRef ref,
UpdateState state,
) async {
if (state.phase == UpdatePhase.available ||
state.phase == UpdatePhase.unsupported ||
state.phase == UpdatePhase.launched) {
await showUpdateDialog(context);
return;
}
await ref.read(updateControllerProvider.notifier).check();
if (!context.mounted) return;
```

```
final phase = ref.read(updateControllerProvider).phase;
if (phase == UpdatePhase.available || phase == UpdatePhase.unsupported) {
await showUpdateDialog(context);
}
}
}

String _subtitle(AppLocalizations l10n, UpdateState state) {
final current = state.current?.text;
final prefix = current == null ? '' : l10n.updateCurrent(current);
final detail = switch (state.phase) {
UpdatePhase.idle || UpdatePhase.checking => l10n.updateChecking,
UpdatePhase.upToDate => l10n.updateUpToDate,
UpdatePhase.available ||
UpdatePhase.downloading ||
UpdatePhase.launched => l10n.updateAvailable(state.release!.version.text),
UpdatePhase.unsupported => l10n.updateUnsupportedShort,
UpdatePhase.failed => _failureText(
l10n,
state.failure ?? UpdateFailure.check,
),
};
if (state.phase == UpdatePhase.downloading) {
return '$detail ${l10n.updateDownloading((state.progress * 100).round())}';
}
return prefix.isEmpty ? detail : '$prefix · $detail';
}

// ===== lib/l10n/l10n.dart =====
import 'package:flutter/widgets.dart';

import '../domain/entities/app_error.dart';
import 'gen/app_localizations.dart';

export 'gen/app_localizations.dart';

extension L10nX on BuildContext {
AppLocalizations get l10n => AppLocalizations.of(this);
}

/// 将领域错误转为用户可读文案
String errorText(AppLocalizations l10n, Object? error) => switch (error) {
RiskControlError(:final message) => message ?? l10n.errorRiskControl,
NeedLoginError() => l10n.errorNeedLogin,
NetworkError() => l10n.errorNetwork,
NotFoundError() => l10n.errorNotFound,
CopyrightError() => l10n.errorCopyright,
UnknownError(:final message) => l10n.errorUnknown(message),
_ => l10n.errorUnknown('$error'),
};
```

```
/// `mm:ss` / `h:mm:ss`
String formatDuration(Duration d) {
final h = d.inHours;
final m = d.inMinutes.remainder(60).toString().padLeft(2, '0');
final s = d.inSeconds.remainder(60).toString().padLeft(2, '0');
return h > 0 ? '$h:$m:$s' : '$m:$s';
}

/// 字节数格式化: 1.2 MB / 3.4 GB
String formatBytes(int bytes) {
const units = ['B', 'KB', 'MB', 'GB'];
var v = bytes.toDouble();
var i = 0;
while (v >= 1024 && i < units.length - 1) {
v /= 1024;
i++;
}
return '${v.toStringAsFixed(i == 0 ? 0 : 1)} ${units[i]}';
}
```